

IBM Module Refactor in Cerisse

GPStore, BVH Acceleration, and Modular Header Split

Auto-generated

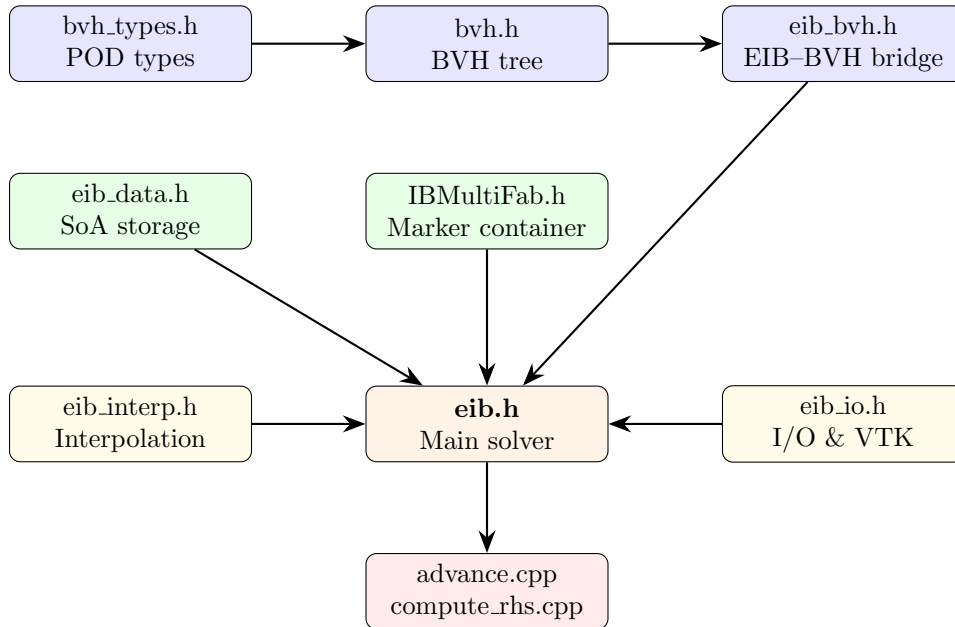
March 2026

1 Motivation

The original `eib.h` was a monolithic ~ 3800 -line header containing geometry I/O, interpolation, surface output, and the main solver class. The refactor aims to:

1. **Split into modules** for maintainability.
2. **Replace CGAL-only** inside/outside and closest-point queries with a GPU-compatible BVH acceleration structure.
3. **Introduce GPStore** — a level-wide CSR-indexed ghost-point storage that enables single-kernel `computeAllGPs()` instead of per-FAB `computeGPs()`.
4. **Support moving geometry** via `rebuildGeometryData()`.

2 Architecture



3 New Files

File	Lines	Purpose
------	-------	---------

<code>bvh_types.h</code>	478	POD type definitions: <code>Point</code> , <code>Vec</code> , <code>AABB</code> , <code>BVHNode</code> , <code>TriMesh</code> (3D), <code>Polygon2D</code> (2D), <code>SurfElem</code> , <code>LocalFrame</code> , <code>ClosestPointResult</code> , <code>BoundedSide</code> enum. All types are GPU-annotated.
<code>bvh.h</code>	524	BVH construction (Morton-code median split on CPU) and query (stack-based traversal on GPU). Supports both <code>TriMesh</code> and <code>Polygon2D</code> geometries. Core functions: <code>build()</code> , <code>closest_point_query()</code> , <code>BVHQueryView</code> (POD capture for GPU lambdas).
<code>eib.bvh.h</code>	942	Bridge between EIB and BVH. Key components: • <code>InsideTester/InsideTesterView</code>: GPU inside/outside classification via 3-ray majority-vote (Möller–Trumbore). • STL reader (ASCII + binary), OFF reader, 2D polygon reader. • <code>build_geometry_cache()</code>: precompute <code>LocalFrame</code> and <code>SurfElem</code> arrays.
<code>eib.data.h</code>	637	Compile-time constants (<code>MAX_NGEOM</code> , interpolation thresholds, image-point scaling factors). SoA data structures: <code>gpData_t</code> (ghost points), <code>surfImp_t</code> (surface image points), <code>surfPhys_t</code> (surface physical data), <code>FaceCSR</code> (compressed sparse row for per-FAB face iteration). SFINAE traits for wall-model dispatch.
<code>eib.interp.h</code>	519	Included inside <code>eib_t</code> class body. Functions: <code>valid_mirror()</code> , <code>search_optimal_image_point()</code> , <code>search_image_point()</code> , <code>computeIPweights()</code> (bi-/tri-linear), <code>interpolateIMs()</code> , <code>extrapolate()</code> (Lagrange, adaptive order), <code>global2local()/local2global()</code> frame transforms.
<code>eib.io.h</code>	544	Included inside <code>eib_t</code> class body. Functions: <code>read_geom()</code> (master geometry loader), <code>plotSURF()</code> (VTK <code>.vtp</code> writer), <code>gatherSurfData()</code> (MPI gather to rank 0), <code>buildCSR()</code> (per-level face CSR).

4 Modified Files

4.1 `eib.h` — Main Solver Class

Reduced from ~3800 to ~1400 lines by extracting helpers into the six new headers above.
Key new members and methods:

Member / Method	Description
<code>gpstore_a[level]</code>	Level-wide flattened ghost-point store (CSR-indexed by FAB). Contains <code>GPStoreView</code> (POD for GPU capture) and <code>GPStore</code> (owns <code>ManagedVector</code>).

<code>computeAllGPs()</code>	Single <code>ParallelFor</code> over all GPs on a level. Uses <code>gpstore_a</code> + device <code>Array4</code> pointers + FAB-offset mapping. Replaces per-FAB <code>computeGPs()</code> .
<code>rebuildGeometryData()</code>	Recomputes BVH trees, inside/outside testers, bounding boxes, and surface-element caches after vertex positions change (moving geometry).
<code>geom_a[ngeom]</code>	Explicit geometry storage (<code>TriMesh</code> or <code>Polygon2D</code>).
<code>bvh_a[ngeom]</code>	BVH acceleration structures per geometry.
<code>inout_fa[ngeom]</code>	<code>InsideTester</code> per geometry for inside/outside queries.
<code>faces_per_level</code>	CSR structure for efficient face iteration per level/-FAB.

4.2 IBMultiFab.h — Marker Container

- Explicit delete of copy constructors (prevent expensive deep-copies of `Gpu::ManagedVector`).
- Default move constructors enabled.
- `gpData` (ghost-point geometry) is NOT carried in alias constructors or `FillBoundary` — it is local per-FAB data.
- Added `copytoRealMF()` for plotfile output.

4.3 ib_walltypes.h — Wall Model Parameters

- Added `interior_is_solid` flag to `ibmparm_t`.
- Extended `compute_surfIB` to 7-argument signature matching the new `eib.h` surface computation interface.
- Added `const int&` helper overload for SFINAE trait detection without call-site ambiguity.

4.4 eib_cgal.h — CGAL Wrappers

- New BVH-based inside/outside testing functions alongside existing CGAL paths.
- Closest-point queries extended for the BVH code path.
- Retained CGAL as optional high-accuracy fallback.

5 GPStore Architecture

5.1 Data Layout

Ghost points are stored in a flat CSR (Compressed Sparse Row) structure per AMR level:

Listing 1: GPStore CSR layout

```

1 struct GPStoreView {    // POD, GPU-capturable
2     int*   gp_fab;       // gp_fab[ii] = local FAB index for GP ii
3     int*   fab_offsets;  // fab_offsets[f] = start index of FAB f
4     // ... per-GP fields (xyz, ijk, normals, weights, etc.)
5 };
6
7 struct GPStore {        // owns Gpu::ManagedVector storage
8     Gpu::ManagedVector<int> gp_fab_v;

```

```

9      Gpu::ManagedVector<int> fab_offsets_v;
10     // ...
11     void allocate(); // resize after gp_counts known
12     GPStoreView view(); // return POD for GPU capture
13 };

```

5.2 Three-Phase Workflow

The GPStore enables a three-phase workflow that replaces the old per-FAB `computeGPs()`:

1. **Phase 1 — cons2prims:** Fill a level-wide `prims_mf` MultiFab from the conservative state (one MFIter loop).
2. **Phase 2 — computeAllGPs:** Single `ParallelFor` over all GPs on the level. Each GP reads from the correct FAB via `gp_fab[ii]` mapping and `fab_offsets` offset.
3. **Phase 3 — prims2cons:** Convert primitives back to conservatives at GP cells only (one MFIter loop, only cells with `ibMarkers(i,j,k,1)`).

This pattern is used in both `advance.cpp` (post-Runge-Kutta correction) and `compute_rhs.cpp` (pre-flux computation).

6 Moving Geometry Support

For moving immersed boundaries, the following sequence is executed at the start of each time step in `advance.cpp`:

Listing 2: Moving geometry update sequence

```

1  #ifdef AMREX_USE_GPIBM
2  if (CNS::ib_move) {
3      // 1. User callback updates vertex positions
4      PROB::update_geometry(time + dt, IBM::ib.geom_a, IBM::ib.ngeom);
5      // 2. Rebuild BVH, inside/outside testers, surface cache
6      IBM::ib.rebuildGeometryData();
7      // 3. Recompute markers and GP data on all levels
8      rebuildIBM();
9  }
10 #endif

```

`rebuildGeometryData()` performs:

1. Rebuild BVH trees from updated vertex positions.
2. Reconstruct `InsideTester` for each geometry.
3. Recompute bounding boxes.
4. Rebuild `LocalFrame` and `SurfElem` caches.

7 Integration with `compute_rhs.cpp`

The main changes in `compute_rhs.cpp` for the IBM refactor:

Change	Detail
--------	--------

Compile guard	<code>#error</code> if both <code>AMREX_USE_GPIBM</code> and <code>CNS_USE_EB</code> are defined.
GPStore pre-loop	Level-wide <code>cons2prims()</code> + <code>computeAllGPs()</code> before the <code>MFilter</code> loop.
Prims aliasing	Inside the loop, <code>prims</code> aliases <code>prims_mf.array(mfi)</code> (no per-FAB allocation when IBM is active).
geoMarkers alias	Direct <code>Array4</code> alias to <code>IBMultiFab</code> markers; eliminates the old auxiliary <code>BaseFab<uint8_t></code> + <code>ParallelFor</code> copy.
Solid RHS zeroing	Uses explicit <code>if (geoMarkers != 0)</code> branch instead of <code>state *= (1 - int(marker))</code> multiplication.

8 Known TODOs

- `computeSURFs` in `postCoarseTimeStep` is commented out — needs re-enabling with GPStore.
- `rz_geometric_source_ibm` is not implemented (RZ + IBM combination).
- “Pure virtual method called” crash at program exit (pre-existing, likely destruction-order issue with AMReX/IBM singletons).